dida-trade

Design and Implementation of Distributed Applications 2026-2027
IST (MEIC-A / MEIC-T / METI)
Project

Version 1.0

1 Introduction

The goal of the dida-trade project is to implement a simple meeting management application that is replicated using the Paxos consensus algorithm. The application is composed of different components, namely:

- Multiple *servers*, that keep the (replicated) state of the application.
- Multiple *clients*, that place orders to buy and sell items.
- A single *console*, that can send configuration commands to the servers.

The project should be developed in different phases, described later in this document.

2 Operation of the dida-trade

The dida-trade is an application for matching sellers and buyers, allowing users to sell or acquire items. There is only one class of items (say, "Nuka Cola Caps") and all items are exactly the same. All users buy, sell and hold the same type of items; only the quantity matters. Note that the application could be easily extended to consider different types of items (for instance, to allow users to exchange *caps* for *stimpacks*), but such extension is irrelevant for the topics covered in this course.

The application keeps information about *users*. Each user has a unique *user identifier*, a *wallet* with some amount of currency and a *stock* of some amount of items. User can sell part of their stock to other users or acquire stock from other users. To simplify the project, it is assumed that each item costs exactly one unit of currency.

As it will be described below, when a user puts a order to sell items it states what *fraction* of their items it wants to trade. This appears to be silly (and it is). However, it

avoids the extra code to address corner-cases (such as the scenario where a user decides to sell more items that it actually owns). Similarly, when a user puts a buy order, it also specifies the fraction of money they wants to spend.

The application accepts 4 different commands. For simplicity, all values (participant identifiers and fractions) are integers.

- POPULATE, that is used to initiate the *users* database. This request accepts a single parameter, n, and creates n users, with identifiers in the range $[0, n[$, each with a wallet of n currency units and a stock of n items.

- ADD a user to the database. This requests accepts three parameters, namely the user unique identifier, the initial value of the wallet and the initial stock of that user.

- SELL that accepts as input parameters a user identifier and a number *frac*, in the range $[0, 100[$, that indicates the fraction (in percentage) of the user's stock it wants to sell (for instance if the user stock is 400 items and $frac = 25$, the user will sell 25% of its stock (i.e., 100 items).

- BUY that accepts as input parameters a user identifier and a number *frac*, in the range $[0, 100[$, that indicates the fraction (in percentage) of the user's wallet it wants to spend to acquire stock (for instance if the user wallet is 200 and $frac = 75$, the user will spend 75% of its wallet (i.e., 150 units of currency to acquire 150 items).

- BALANCE that accepts as input parameter a user identifier and returns the user's owned amount of currency.

Requests are totally ordered by the system. A request that is accepted returns as soon as it is placed. For instance, if the user places a request to acquire x items but there are no sellers at that point, the request is accepted and returns immediately. In background, the items will be added to the user stock when other users decide to sell their stock. Items can be sold/acquired to/from different users (for instance, if a seller decides to sell 10 items, 4 items can be sold to one user and 6 items to another user).

Sell and buy requests are totally ordered and served in that order. When a "sell" order starts to be served, no other "sell" order is served until that order is complete. For instance, assume that user u_1 puts a request to sell 10 items and user u_2 puts a request to sell 5 items. If the request from u_1 is ordered before the request from u_2, u_2 will need to wait for u_1 to sell all 10 items before being able to sell their own items.

The same policy is used for "buy" orders.

In the project's early stages, BALANCE should be totally ordered. In later steps (project steps are discussed below), students may implement a version of BALANCE that respects causal order only.

3 Replication Algorithm

3.1 State-machine Replication and Paxos

The project implements *state-machine replication*. In this algorithm, all requests from clients are accepted in total order. To implement this algorithm, the system will run multiple instances of consensus. The first instance will decide which is the first request to be executed by the state machine, the second instance decides which is the second request to be executed by the state machine, etc.

To implement state-machine replication, replicas must agree on a total order to execute commands. To achieve this goal, the project will use the Paxos algorithm. Paxos is a consensus algorithm that ensures safety, even in periods of asynchrony. When executing Paxos, participants may have different roles, denoted as *proposers*, *acceptors*, and *learner*. Paxos will be discussed in detail in classes.

3.2 Multi-Paxos

The simplest way of using Paxos is to decide one command at a time, i.e., an instance of Paxos is executed to decide which request is executed first and only when this instance is terminated and a request has been decided a new Paxos is started (to decide the next request).

Multi-Paxos is a variant of Paxos that allows a new instance to be started before the previous instance is terminated. More precisely, when using Multi-Paxos, a leader may propose some value v' for instance n immediately after proposing a value v for instance $n-1$, without waiting for v to be first accepted by a majority of acceptors. This allows the system to offer better throughput.

3.3 Fast-Paxos

Fast Paxos is a variant of Paxos that, in good runs, allows consensus to be achieved with lower latency (than when using the "standard") Paxos algorithm.

3.4 Requests Without Total Order

In this variation, BALANCE operations are not totally ordered and can be processed by different replicas in different orders. The operation must respect causal order and reflect all the operations previously observed by the client.

3.5 Dynamic Membership

Most of the project can be executed using just 3 replicas, a configuration that can tolerate the failure of 1 replica. The project may also support dynamic membership, where some replicas can be eliminated and new replicas can be added.

4 Servers, Replicas and Configurations

There are n servers. Each server is identified by a number in the range $0, 1, \ldots, (n-1)$. As a simplification, we assume that the number of servers is fixed during a given execution. To further simplify the project, it is assumed that the IP address and port p of the server 0 is well known by all clients and servers (it is passed as a parameter when a process starts) and that the other servers run on consecutive ports in the same machine (i.e., server i runs on port $p + i$).

Clients send requests to all servers and return as soon as they receive a reply.

Also as a simplification, we will execute all processes (servers and clients) on the same machine. Naturally, a real system would run nodes in different machines.

We also assume that all servers are running when clients start. All servers always act as Paxos "learners" (L), i.e., they receive requests from clients, observe the order assigned to requests, execute the requests, and may reply to clients. However, at any given point in time, only a subset of the servers may act as "proposers" (P) and "acceptors" (A). This set of servers is defined as a *configuration*.

The configuration is also used to define if a proposer will use "Fast-Paxos round" or "Standard Paxos" round. If a proposer is configured to use Fast Paxos, this is tagged with the character '`F`'.

To manage configurations we will use an approach known as "Vertical Paxos". In this case, the configuration specifies the role of each server. In each configuration, only one of the proposers is selected as a leader (this server is marked as "P"). Each configuration is associated with a Paxos ballot number and the leader that is active in that configuration uses the ballot number associated with the configuration. To simplify the project, we assume two different strategies (schedules) to manage the configuration of the servers. Vertical Paxos is used in all phases of the project.

Note that Vertical Paxos supports the use of different quorums in each phase of Paxos. To simplify, the project will always use a majority of acceptors as a quorum.

4.1 Schedule A

In this schedule, 3 servers S_0, S_1, S_2 are used. These servers are both proposers, acceptors, and learners.

In this schedule, there is a infinite sequence of configurations, where servers S_0, S_1, S_2 change the role as leaders (i.e., proposers that are active as leaders in that configuration)

in a round-robin manner. Server S_0 is the leader for $Config_0$, $Config_3$, $Config_6$, etc. Server S_1 is the leader for $Config_1$, $Config_4$, $Config_7$, etc. Server S_2 is the leader for $Config_2$, $Config_5$, $Config_8$, etc.

Configurations are in the form of a dictionary with a tuple per server in the system $\{(server_{id}, roles), ...\}$, where $server_{id}$ is its identifier and $roles$ is a string representing one or more roles the server plays in the Paxos configuration. The $roles$ string can have up to four characters:

- F, marking the use of Fast-Paxos in the configuration;
- P, marking the server as a Paxos Proposer;
- A, marking the server as a Paxos Acceptor;
- L, marking the server as a Paxos Learner;

The scheduling can be found bellow:

- $Config_0$: $\{(0, FPAL), (1, AL), (2, AL)\}$.
- $Config_1$: $\{(0, AL), (1, PAL), (2, AL)\}$.
- $Config_2$: $\{(0, AL), (1, AL), (2, PAL)\}$.
- $Config_3$: $\{(0, FPAL), (1, AL), (2, AL)\}$.
- $Config_4$: $\{(0, AL), (1, PAL), (2, AL)\}$.
- $Config_5$: $\{(0, AL), (1, AL), (2, PAL)\}$
- $Config_6$: $\{(0, FPAL), (1, AL), (2, AL)\}$.
- ...

For instance, in $Config_0$, server 0 is the leader proposer, servers $\{0, 1, 2\}$ are acceptors, and servers $\{0, 1, 2\}$ are learners.

The system starts in $Config_0$ and can dynamically change to the next configuration. Note that it is possible that a server moves to configuration j while another server is sill using configuration $i < j$.

4.2 Schedule B

Same has before, but all configurations use Fast-Paxos. I.e.:

- *Config*$_0$*:* $\{(0, FPAL), (1, AL), (2, AL)\}$.
- *Config*$_1$*:* $\{(0, AL), (1, FPAL), (2, AL)\}$.
- *Config*$_2$*:* $\{(0, AL), (1, AL), (2, FPAL)\}$.
- *Config*$_3$*:* $\{(0, FPAL), (1, AL), (2, AL)\}$.
- *Config*$_4$*:* $\{(0, AL), (1, FPAL), (2, AL)\}$.
- *Config*$_5$*:* $\{(0, AL), (1, AL), (2, FPAL)\}$
- *Config*$_6$*:* $\{(0, FPAL), (1, AL), (2, AL)\}$.
- ...

4.3 Schedule C

In this schedule, 7 servers are used. There are only 5 possible configurations, and the set of acceptors changes from *Config*$_2$ to *Config*$_3$.

- *Config*$_0$*:* $\{(0, FPAL), (1, AL), (2, AL), (3, AL), (4, AL), (5, L), (6, L)\}$.
- *Config*$_1$*:* $\{(0, AL), (1, PAL), (2, AL), (3, AL), (4, AL), (5, L), (6, L)\}$.
- *Config*$_2$*:* $\{(0, L), (1, L), (2, AL), (3, FPAL), (4, AL), (5, AL), (6, AL)\}$.
- *Config*$_3$*:* $\{(0, L), (1, L), (2, AL), (3, AL), (4, PAL), (5, AL), (6, AL)\}$.
- *Config*$_4$*:* $\{(0, L), (1, L), (2, AL), (3, AL), (4, AL), (5, PAL), (6, AL)\}$.

4.4 Changing Configurations

The project considers two different mechanisms for changing the configuration used by Paxos.

- *Manual Reconfiguration:* The configuration is explicitly and manually changed by the console. The console sends a command to the replicas to instruct replicas to start executing using a given configuration. The leader of that configuration will then initiate the steps required by Paxos. Note that it is usually enough to inform the leader of the configuration. The other replicas will become aware of the configuration change when they receive messages from the leader of that configuration.

- *Automatic Reconfiguration:* If the system is in reconfiguration i and, in this configuration, Fast Paxos is used, any process in configuration i can automatically start executing in configuration $i + 1$ if it realizes that no value may have been decided in configuration i (this is possible because all processes are learners).

5 Console

The *console* is used to send commands to control the behavior of servers. The console can issue the following commands:

- *ballot (ballot_number, replica_id):* informs the replica `replica_id` that ballot with number `ballot_number` has been initiated /this message is typically sent to the leader of that ballot, such that it initiates the first step of Paxos with that ballot number).
- *activate (ballot_number, replica_id):* activates the leader of `ballot_number` (this is used in one of the variants of Vertical pPaxos (known as Vertical Paxos II), that will be discussed in classes).
- *debug (mode, replica_id):* that triggers some pre-configured debug behavior `mode` on a given server replica `replica_id`.

The following debug modes should be implemented. Students are free to implement other debug modes to test their algorithms.

- crash: the server crashes (i.e., exits).
- freeze: the server "freezes", i.e., blocks all requests from clients and from other servers (except from the console) until it is "un-freezes". The blocked requests should not be dropped, and should be resumed when the server receives the un-freeze command.
- un-freeze: the server "un-freezes", i.e., processes all pending blocked requests from clients and removes the blocking condition.
- slow-mode-on: the server applies a random delay before it processes any request from the client or from other servers (except requests from the console).
- slow-mode-off: the server stops applying a random delay to requests.

6 Programming Language and Background

The project should be implemented in Java and GRPC. IST students should have the necessary background to execute the project, as it builds on the material covered by the Distributed Systems course at the BSc level. Students that are not familiar with Java and GRPC will need to learn the required background using self study and by doing the exercises prepared for the Distributed Systems course (that are available online here).

7 Project Steps

It is recommended to develop the project in a stepwise manner. The following steps should be followed:

- *Step 0 (provided): Baseline.* In this implementation, the system implements state-machine replication and runs "Schedule A" only. A simple, and incomplete, version of the baseline has been already implemented and is provided to students.
- *Step 1: Complete the baseline.* Implement debug modes required to show that the baseline is bogus. Complete the baseline, namely by doing a proper Paxos "phase 1".
- *Step 2: Multi-Paxos.* The system implements Multi-Paxos, running "Schedule A".
- *Step 3: Fast-Paxos.* The system implements Fast-Paxos, running "Schedule A".
- *Step 4: Causally ordered* BALANCE. Same as above, but implementing BALANCE without using Paxos.
- *Step 5: Dynamic Membership.* The system implements "Schedule B". This involves performing a state transfer, such that the new replicas have knowledge about values decided in the past.

8 Final Report

Students should prepare a final report describing the developed solution (max. 6 pages). In this report, students should follow the typical approach of a technical paper, first describing very briefly the problem they are going to solve, the proposed implementation solutions, and the relative advantages of each solution. Please avoid including in the report any information already mentioned in this project description. The report should include an explanation of the algorithms used and justifications for the design decisions. The final reports should be written using LaTeX. A template of the paper format will be provided to the students.

9 Checkpoint and Final Submission

The grading process includes an intermediate optional checkpoint and a mandatory final submission. For the checkpoint the students may submit a preliminary implementation of the project; if they do so, the checkpoint grade may improve their final grade. The goal of the checkpoint is to control the evolution of the implementation effort.

The final submission should include the source code (in electronic format) and the associated report (max. 6 pages).

10 Project Goals

The students can set their own goals for the project. Depending on the goals achieved, there is a ceiling to the top grade they can receive, as follows:

- *Basic:* Execute Step 1 by the checkpoint and Step 2 by the final delivery date. Max grade: 14/20.
- *Vanilla:* Execute Step 1 and Step 2 by the checkpoint and Step 3 by the final delivery date. Max grade: 16/20.
- *Premium:* Execute Step 1 and Step 2 by the checkpoint and Step 3 and Step 4 by the final delivery date. Max grade: 18/20.
- *Ultimate:* Same as "premium" plus Step 5. Max grade: 20/20.

11 Relevant Dates

- September 30th - Electronic submission of the checkpoint code;
- October 16th - Electronic submission of the final code and report.

12 Grading

The project grading will depend on a discussion at the end of the semester where all members of the groups must be present and where individual grades will be determined. That grade will depend on, besides the quality of the project, the individual performance in the discussion and the lecturer's evaluation.

The project grade (45% of the course's grade) is the *best* of the following two:

- Final_Project_Grade
- 85% of the Final_Project_Grade + 15% of Checkpoint_Grade

13 Use of AI

The following rules apply to the use of AI in the project.

- Level 1 (yellow) – Limited use, with the following declaration:

  "Todas as decisões e conteúdo técnico deste trabalho são da minha autoria. Foram utilizadas [listar ferramentas] para [funções]"

  "*All design choices and the code have been by us. The following tools [list the tools] have been used for this purpose [list the purpose].*"

14 Authorship

The following declaration must be included in the report:

```
We,

[Name 1], n.º [xxxx], course
[Name 2], n.º [xxxx], course
[Name 3], n.º [xxxx], course

We hereby declare, on our honor, that the work entitled "[Title of the Work]", carrie

[Name 1] { [summary of contribution]
[Name 2] { [summary of contribution]
[Name 3] { [summary of contribution]

...
We declare that this work is original and complies with the applicable academic regul

[Local], [Date]

Signatures:
```

15 Cooperation among Groups

Students must not, *in any case*, see the code of other groups or provide their code to other groups. If copies of code are detected, all groups involved will fail the course.

16 "Época especial"

Students being evaluated during the "Época especial" will be required to do a different project and an exam. The "Época especial" project will be announced on the first day of the "Época especial" period, must be delivered on the day before last of that period, and will be discussed on the last day.